

# Resumen primer parcial

## Máquinas de Turing

### Tipos de problemas con MT

- Problema de **búsqueda**: La MT devuelve una respuesta (cadena, número, etc.) o no devuelve nada (rechazo, estado  $q_R$ ).
- Problema de **decisión**: La MT responde sí (aceptación, estado  $q_A$ ) o no (rechazo, estado  $q_R$ ).

### Máquina de Turing (MT)

- **Cinta** infinita dividida en celdas, cada una con un símbolo de un alfabeto finito.
- **Cabezal** que se mueve un paso a la izquierda o derecha, o se queda en la misma posición, y que puede leer y escribir símbolos en la cinta.
- **Estados** finitos, incluyendo un estado inicial, estados de aceptación y estados de rechazo.
- La MT cambia de estado en cada paso según una función de transición que depende del estado actual y del símbolo que está leyendo el cabezal.
- En un paso una MT puede modificar el símbolo que está leyendo, cambiar de estado y moverse un lugar a la izquierda o a la derecha, o quedarse en la misma posición.
- Al iniciar, la cadena de entrada se delimita por infinitos blancos a ambos lados y el cabezal apunta al primer símbolo de la cadena. El estado inicial siempre es  $q_0$ .
- **Tesis de Church-Turing**: Todo lo que es computable por un algoritmo es computable por una MT.

### Modelos equivalentes de MT

- Dos MT son equivalentes si aceptan el mismo lenguaje.
- Dos modelos de MT son equivalentes si dada una MT de un modelo se puede construir una MT del otro modelo que sea equivalente a la primera.
- Algunos modelos equivalentes al modelo estándar de MT con una cinta:
  - MT con  $k$  cintas.
  - MT con cintas semi-infinitas.
  - MT con 2 cintas pero un solo estado.
  - MT no determinísticas.
- Ejemplos de modelos computacionales equivalentes al modelo de MT:
  - Máquinas RAM.
  - Circuitos booleanos.
  - Cálculo lambda.
  - Funciones recursivas parciales.

- Gramáticas.
- Programas Java.
- etc.
- Todo lo anterior refuerza la Tesis de Church-Turing.

## Otros tipos de MT

### MT calculadora

- Se usa para problemas de búsqueda.
- Realiza un cálculo, generalmente matemático, y devuelve un resultado.
- Además del estado en el cual termina la máquina, importa qué es lo que deja en la cinta, ya que ese es el resultado del cálculo.

### MT generadora

- Se trata de una MT que genera, una por una, todas las cadenas de un lenguaje en particular.
- En general la generación se hace en orden canónico (primero las cadenas más cortas, luego las más largas).
- **Teorema:** Existe una MT  $M$  que acepta un lenguaje  $L$  si y solo si existe una MT generadora  $M'$  que genera  $L$ .
  - Si  $M$  acepta  $L$ , entonces  $M'$  genera  $L$ .
  - Si  $M'$  genera  $L$ , entonces  $M$  acepta  $L$ .

## Jerarquía de Computabilidad

### Lenguajes recursivos ( $R$ )

- Un lenguaje  $L$  es **recursivo** ( $L \in R$ ) si y solo si existe una MT  $M$  que lo acepta y siempre se detiene.
  - Si  $w \in L$ , entonces  $M$  a partir de  $w$  se detiene en el estado de aceptación  $q_A$ .
  - Si  $w \notin L$ , entonces  $M$  a partir de  $w$  se detiene en el estado de rechazo  $q_R$ .

### Lenguajes recursivamente enumerables ( $RE$ )

- Un lenguaje  $L$  es **recursivamente enumerable** ( $L \in RE$ ) si y solo si existe una MT  $M$  que lo acepta.
  - Si  $w \in L$ , entonces  $M$  a partir de  $w$  se detiene en el estado de aceptación  $q_A$ .
  - Si  $w \notin L$ , entonces  $M$  a partir de  $w$  se detiene en el estado de rechazo  $q_R$  o no se detiene.

## Propiedades de la clase $R$

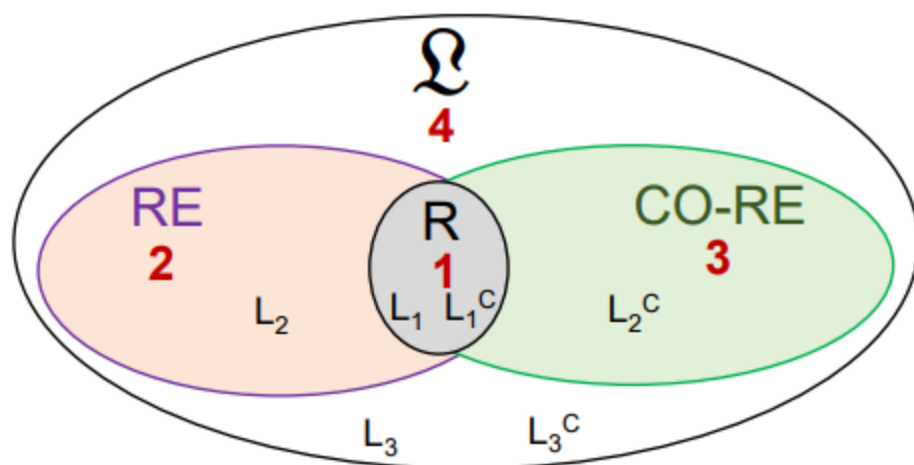
1. Si  $L \in R$  entonces  $L^C \in R$ , con  $L^C = \Sigma^* - L$ . Demostración breve:
  - i. Dada una MT  $M_L$  que acepta  $L$  y siempre se detiene, se construye una MT  $M_{L^C}$  que acepta  $L^C$  y siempre se detiene.

- ii.  $M_{L^C}$  es igual a  $M_L$  excepto que los estados de aceptación y rechazo se intercambian. Es decir, todas las transiciones que van al estado de aceptación  $q_A$  de  $M_L$  van al estado de rechazo  $q_R$  de  $M_{L^C}$ , y todas las transiciones que van al estado de rechazo  $q_R$  de  $M_L$  van al estado de aceptación  $q_A$  de  $M_{L^C}$ .
  - iii. Así,  $L^C \in R$ .
2. Si  $L_1 \in R$  y  $L_2 \in R$  entonces  $L_1 \cap L_2 \in R$ . Demostración breve:
- i. Dadas dos MT  $M_1$  y  $M_2$  que aceptan  $L_1$  y  $L_2$  respectivamente y siempre se detienen, se construye una MT  $M$  que acepta  $L_1 \cap L_2$  y siempre se detiene.
  - ii.  $M$  ejecuta secuencialmente primero  $M_1$  y luego  $M_2$ . Si  $M_1$  acepta, entonces  $M$  ejecuta  $M_2$ . Si  $M_2$  acepta, entonces  $M$  acepta. En cualquier otro caso,  $M$  rechaza. Como tanto  $M_1$  como  $M_2$  siempre se detienen,  $M$  también siempre se detiene.
  - iii. Así,  $L_1 \cap L_2 \in R$ .

## Propiedades de la clase $RE$

1. Si  $L_1 \in RE$  y  $L_2 \in RE$  entonces  $L_1 \cup L_2 \in RE$ . Demostración breve:
- i. Dadas dos MT  $M_1$  y  $M_2$  que aceptan  $L_1$  y  $L_2$  respectivamente, se construye una MT  $M$  que acepta  $L_1 \cup L_2$ .
  - ii.  $M$  ejecuta simultáneamente (en paralelo)  $M_1$  y  $M_2$ . Si cualquiera de las dos máquinas acepta, entonces  $M$  acepta. En cualquier otro caso,  $M$  no se detiene.
  - iii. Así,  $L_1 \cup L_2 \in RE$ .

## Diagrama de Venn de la jerarquía de computabilidad



Las cuatro regiones de la jerarquía,  
con grado de dificultad creciente.

1. **Conjunto  $R$ .**
  - i. Lenguajes aceptados por MT que siempre paran.
  - ii. Si  $L$  está en  $R$ , entonces  $L^C$  también está en  $R$ .
2. **Conjunto  $RE - R$ .**
  - i. Lenguajes aceptados por MT que no siempre paran.
  - ii. Si  $L$  está en  $RE$ , entonces  $L^C$  está en  $CO-RE$ .
3. **Conjunto  $CO-RE - R$ .**

- i. Lenguajes que no son aceptados por ninguna MT, pero sus complementos sí son aceptados por alguna MT.
  - ii. Si  $L$  está en CO-RE, entonces  $L^C$  está en RE.
4. **Conjunto  $\mathcal{L} = (RE \cup \text{CO-RE})$ .**
- i. Lenguajes que no son aceptados por ninguna MT, ni tampoco sus complementos son aceptados por alguna MT.
  - ii. Si  $L$  está en  $\mathcal{L} = (RE \cup \text{CO-RE})$ , entonces  $L^C$  también está en  $\mathcal{L} = (RE \cup \text{CO-RE})$ .

## Diagonalización

### El lenguaje $D$

- Se tiene un lenguaje  $D \in RE - R$  que se usa para probar que  $R \subset RE$ .
- Se tiene un lenguaje  $D^C \in \text{CO-RE} - R$  que se usa para probar que  $RE \subset \mathcal{L}$ .
- El proceso es el siguiente:
  - Se prueba de forma constructiva que  $D \in RE$ .
  - Se prueba de forma no constructiva por diagonalización que  $D^C \notin RE$ .
  - Finalmente se concluye que  $D \notin R$ , porque si  $D \in R$  entonces  $D^C \in R$  cosa que es absurdo porque se sabe que  $D^C \notin RE$ .

### Detalle de la diagonalización

- Se tiene una tabla donde cada fila corresponde a una MT  $M_i$  y cada columna corresponde a una cadena  $w_j$ .
- En cada celda  $(i, j)$  se pone un 1 si  $M_i$  acepta  $w_j$ , y un 0 si  $M_i$  rechaza  $w_j$  ya sea porque se detiene en el estado de rechazo  $q_R$  o porque no se detiene.
- La diagonal de la tabla representa el lenguaje  $D = \{w_i \mid M_i \text{ acepta } w_i\}$
- La diagonal de la tabla con los bits invertidos representa el lenguaje  $D^C = \{w_i \mid M_i \text{ rechaza } w_i\}$
- Se cumple que  $D \in (RE - R)$  y  $D^C \in (\text{CO-RE} - R)$ .

### MT universal

- Una MTU es una MT capaz de ejecutar otra.
- El esquema más general de una MTU es el siguiente:
  - La MTU recibe como cadena de entrada una tupla  $(\langle M \rangle, w)$  donde  $M$  es la codificación binaria de una MT y  $w$  es una cadena de entrada.
  - La MTU ejecuta a la MT  $M$  a partir de la cadena  $w$ .

### Codificación y enumeración de MT

- Cada MT se puede codificar como una cadena binaria, es decir, cada MT tiene una codificación única  $\langle M \rangle$ .
- Se puede enumerar todas las MTs como  $M_1, M_2, M_3, \dots$ , donde  $\langle M_i \rangle$  es la codificación de la MT  $M_i$ .
- Esta enumeración se hace en el **orden canónico**:
  - De menor a mayor longitud.

- Con longitudes iguales se sigue el orden alfanumérico.
- Para obtener la MT  $M_i$  según el orden canónico se puede usar el siguiente algoritmo:
  - $n = 0$
  - Genera la siguiente cadena  $w$  según el orden canónico.
  - Si  $w$  no es la codificación válida de una MT (esto se chequea fácilmente de forma sintáctica) vuelve al paso 2.
  - Si  $n = i$ , devuelve  $w$ , que es el código de la MT  $M_i$ , y se detiene. Si  $n \neq i$ , hace  $n = n + 1$  y vuelve al paso 2.

## Halting Problem

- El **Halting Problem** ( $HP$ ) postula: dada una MT  $M$  y una cadena de entrada  $w$ , ¿ $M$  se detiene a partir de  $w$ ?
- Formalmente,  $HP = \{(\langle M \rangle, w) \mid M \text{ se detiene a partir de } w\}$
- $HP \in (RE - R)$ . Se prueba por diagonalización.

## Formas de burlar al Halting Problem

- Si bien el Halting Problem es indecidible, con ciertas restricciones al problema como tal se vuelve decidible.

## Detección de loop

- Si una MT se mueve en un espacio de celdas limitado, se puede detectar cuándo entra en loop.
- Supongamos que  $M$  se mueve en no más de  $k$  celdas. ¿Por cuántas configuraciones distintas puede pasar  $M$  antes de loopear?
- Una configuración de una MT tiene:
  - Una posición.
  - Un estado.
  - Un contenido.
- Si  $M$  tiene  $|Q|$  estados y  $|\Gamma|$  símbolos, entonces hará a lo sumo  $k \cdot |Q| \cdot |\Gamma|^k$  pasos antes de repetir una configuración.
  - $k$  posiciones.
  - $|Q|$  estados.
  - $|\Gamma|^k$  contenidos posibles.
- Por lo tanto, se puede detectar si una MT loopea ejecutándola y llevando la cuenta de sus pasos. Si llega a  $k \cdot |Q| \cdot |\Gamma|^k$  pasos sin detenerse, entonces se concluye que  $M$  loopea.

## Detección de aceptación de al menos una cadena

- ¿Cómo detectar si una MT  $M$  acepta al menos una cadena?
- Tenemos que tener precaución en cómo construimos la MT  $M'$  que chequea si  $M$  acepta al menos una cadena. No sirve que  $M'$  ejecute  $M$  con todas las cadenas posibles en orden canónico, porque si  $M$  no se detiene con alguna cadena, entonces  $M'$  tampoco se detendrá y nunca dará una respuesta.
- Solución:
  - $i = 1$
  - Ejecutar  $i$  pasos de  $M$  sobre TODAS las cadenas de longitud a lo sumo  $i$ .
  - Si  $M$  acepta alguna de esas cadenas, entonces  $M'$  acepta.

- iv. Si no, entonces  $i = i + 1$  y se vuelve al paso 2.
- Por ejemplo, si la primer cadena que  $M$  acepta es de 80 símbolos y  $M$  la acepta en 120 pasos, entonces cuando  $M'$  ejecute 120 pasos de  $M$  sobre todas las cadenas de a lo sumo 120 símbolos la encontrará y aceptará.

## Computabilidad en relación al tamaño de un lenguaje

- La computabilidad de un lenguaje tiene más que ver con su definición que con su tamaño.
- Por ejemplo, el lenguaje  $\Sigma^*$  es el "más grande" posible, y sin embargo es decidable ("fácil"), mientras que  $HP$  es un lenguaje mucho más pequeño que  $\Sigma^*$ , pero no es decidable ("difícil").

## Reducciones

### Definición

- Dados dos lenguajes  $L_1$  y  $L_2$ , supongamos que existe una MT  $M_f$  que computa una función  $f : \Sigma^* \rightarrow \Sigma^*$  tal que:
  - A partir de todo  $w \in L_1$ ,  $M_f$  devuelve  $f(w) \in L_2$ .
  - A partir de todo  $w \notin L_1$ ,  $M_f$  devuelve  $f(w) \notin L_2$ .
- Entonces se dice que la función  $f$  es una **reducción** de  $L_1$  a  $L_2$ , se denota como  $L_1 \leq L_2$ , y se dice que  $f$  es **total computable** porque se computa sobre todas las cadenas.

## Recordatorio de la equivalencia lógica contrarrecíproca

- $p \rightarrow q$  es equivalente a  $\neg q \rightarrow \neg p$ .

## Teoremas de las reducciones

1. Si  $L_1 \leq L_2$  y  $L_2 \in R$ , entonces  $L_1 \in R$ .
  - i. También, por contrarrecíproco, si  $L_1 \leq L_2$  y  $L_1 \notin R$ , entonces  $L_2 \notin R$ .
2. Si  $L_1 \leq L_2$  y  $L_2 \in RE$ , entonces  $L_1 \in RE$ .
  - i. También, por contrarrecíproco, si  $L_1 \leq L_2$  y  $L_1 \notin RE$ , entonces  $L_2 \notin RE$ .
3. De forma más resumida:
  - i. Si  $L_1 \leq L_2$  y  $L_1$  es difícil ( $L_1 \notin R$ ) entonces no puede ocurrir que  $L_2$  sea fácil ( $L_2 \in R$ ).
  - ii. Si  $L_1 \leq L_2$  y  $L_1$  es muy difícil ( $L_1 \notin RE$ ) entonces no puede ocurrir que  $L_2$  sea fácil o medianamente difícil ( $L_2 \in RE$ ).
  - iii.  $L_2$  es tan o más difícil que  $L_1$ , porque resolviendo  $L_2$  se resuelve  $L_1$ .
  - iv. De esta manera, las reducciones permiten hallar lenguajes dentro y fuera de  $R$  y  $RE$ .

## Ejemplo $HP \leq L_U$

- $HP = \{(\langle M \rangle, w) \mid M \text{ se detiene a partir de } w\}$
- $L_U = \{(\langle M \rangle, w) \mid M \text{ acepta } w\}$

- De acuerdo al teorema, si  $HP \leq L_U$  y  $HP \notin R$ , entonces  $L_U \notin R$ .
- Por ende, como se sabe que  $HP \notin R$ , si se logra probar que  $HP \leq L_U$  entonces se concluye que  $L_U \notin R$ .

**Reducción:**

- **Definición:**
  - $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w)$  con  $M_2$  igual a  $M_1$  pero con las transiciones hacia el estado  $q_R$  cambiadas por transiciones hacia el estado  $q_A$ .
- **Computabilidad:**
  - Existe una MT  $M_f$  que computa la función  $f$ : copia  $(\langle M_1 \rangle, w)$  pero cambiando los estados  $q_R$  de  $M_1$  por estados  $q_A$  en la codificación de  $M_2$ .
- **Correctitud:**
  - Si  $(\langle M_1 \rangle, w) \in HP$ , entonces  $M_1$  se detiene a partir de  $w$ . Como las transiciones hacia  $q_A$  de  $M_1$  se mantienen en  $M_2$ , entonces  $M_2$  acepta  $w$ . Por lo tanto,  $(\langle M_2 \rangle, w) \in L_U$ .
  - Si  $(\langle M_1 \rangle, w) \notin HP$ , entonces  $M_1$  no se detiene a partir de  $w$ . Tenemos dos casos:
    - Si  $\langle M_1 \rangle$  no es una codificación válida de MT entonces  $\langle M_2 \rangle$  tampoco. Por lo tanto  $(\langle M_2 \rangle, w) \notin L_U$ .
    - Si  $\langle M_1 \rangle$  es una codificación válida de MT, entonces  $M_2$  no se detiene a partir de  $w$ . Por lo tanto,  $(\langle M_2 \rangle, w) \notin L_U$ .
- Por lo tanto queda demostrado que  $HP \leq L_U$  y por lo tanto  $L_U \notin R$ .

## Propiedades de las reducciones

1. **Reflexividad:** Para todo lenguaje  $L$  se cumple  $L \leq L$  usando la función de reducción identidad  $f(w) = w$ .
2. **Transitividad:** Si  $L_1 \leq L_2$  y  $L_2 \leq L_3$  entonces  $L_1 \leq L_3$ . Demostración breve:
  - i. Si  $L_1 \leq L_2$  entonces existe una función de reducción  $f$  de  $L_1$  a  $L_2$ .
  - ii. Si  $L_2 \leq L_3$  entonces existe una función de reducción  $g$  de  $L_2$  a  $L_3$ .
  - iii. Se construye una función de reducción  $h$  de  $L_1$  a  $L_3$  tal que  $h(w) = g(f(w))$ .
  - iv. Por lo tanto,  $L_1 \leq L_3$ .
3. **Simetría:** No se cumple. Si  $L_1 \leq L_2$  no implica que  $L_2 \leq L_1$ .
4. **Complemento:**  $L_1 \leq L_2$  si y solo si  $L_1^C \leq L_2^C$ . Se usa exactamente la misma función de reducción  $f$  para ambas reducciones.

## Autómatas finitos (AF)

- Una cinta readonly.
- Sólo se mueven hacia la derecha.
- Conjunto  $F$  de estados finales.
- Una vez se llega al símbolo blanco el AF para y acepta si y solo si el estado alcanzado es un estado final  $q \in F$ .
- Siempre se detienen.
- Aceptan un tipo limitado de cadenas ya que no tienen memoria.
- Los lenguajes que aceptan se denominan **lenguajes regulares**.
- Se usan para:
  - Análisis sintáctico a nivel palabra de compiladores.
  - Inspecciones de código para QA.

## Autómatas con pila (AP)

- Una cinta de input readonly.
- Una cinta read/write que se comporta como una pila (LIFO).
- En un mismo paso se pueden procesar ambas cintas.
- En la cinta de input el cabezal siempre se mueve hacia la derecha.
- Cuando se alcanza el símbolo blanco en la cinta de input, el AP para, y acepta si y solo si la pila está vacía.
- Siempre se detienen.
- Los lenguajes que aceptan se denominan **lenguajes libres de contexto**.
- Se usan para:
  - Análisis sintáctico a nivel instrucción de compiladores.
  - Evaluación de expresiones en la ejecución de programas.

## Complejidad temporal

### Definiciones

- Una MT tarda más tiempo en terminar (llegar a un estado de aceptación o rechazo) a medida que la longitud de la cadena de entrada, denotada  $|w|$ , aumenta.
- Por esto se usan funciones temporales  $T(n)$  que se definen en términos de la longitud de la cadena de entrada  $n = |w|$ .
- Funciones temporales típicas:  $5n, 3n^3, 2^n, n^{\log_2 n}, 7^{\sqrt{n}}, n!, 6^{n^5}$
- Hay dos grandes grupos de funciones de acuerdo al nivel de abstracción pretendido:
  - Polinomiales o  $poly(n): T(n) = c \cdot n^k$
  - Exponenciales o  $exp(n): T(n) = c \cdot k^n$
- La convención respaldada por las matemáticas y la experiencia es que el **tiempo tratable** es el tiempo polinomial, mientras que el tiempo exponencial es **intratable**.

### Orden $O$

- Una MT tarda tiempo  $T(n)$  si y solo si a partir de toda cadena de entrada  $w$  con  $|w| = n$ , la MT se detiene en a lo sumo  $T(n)$  pasos.
- Una función  $T_1(n)$  es del orden de una función  $T_2(n)$ , denotado  $T_1(n) = O(T_2(n))$ , si y solo si para todo  $n \geq n_0$  se cumple que  $T_1(n) \leq c \cdot T_2(n)$ , con  $c > 0$ .
- Ejemplos:
  - $5n^3 + 8n + 25 = O(n^3)$
  - $n^2 = O(n^3)$
  - $n^3 = O(2^n)$

## Clase $TIME$

- Un lenguaje  $L$  pertenece a la clase  $TIME(T(n))$  si y solo si existe una MT  $M$  que lo decide en tiempo  $O(T(n))$ .
- Para todo lenguaje  $L_i \in TIME(T(n))$  existe una MT  $M_i$  que lo decide en tiempo  $O(T(n))$ .
- Se considera siempre el peor caso (cota superior). Por ejemplo, si a partir de casi todas las cadenas de un lenguaje el tiempo de ejecución es  $O(n)$  pero a partir de algunas pocas cadenas es de  $O(n^3)$ , entonces se dice que el lenguaje pertenece a  $TIME(n^3)$ . Es decir, el tiempo queda "castigado" por las cadenas más difíciles.
- Naturalmente, sería mejor considerar el tiempo promedio o mínimo (cota inferior), pero son mucho más difíciles de calcular y no se usan tanto.

## Clase $P$

- En particular,  $TIME(n^k)$  denota la clase de lenguajes decidibles por una MT en tiempo polinomial de grado  $k$ . También se conoce como la clase  $P$ .
- Todo lenguaje de  $P$  es aceptado por una MT en tiempo  $O(n^k)$ .
- **Robustez:** Si una MT  $M_1$  con  $k$  cintas tarda tiempo  $O(n^k)$ , entonces existe una MT  $M_2$  equivalente con  $c$  cintas que tarda también tiempo  $O(n^k)$ . El retardo es a lo sumo cuadrático.
- Ejemplo de lenguaje en  $P$ :
  - $L_{pal} = \{w \in \Sigma^* \mid w \text{ es un palíndromo con símbolos a y b}\}$
  - MT que lo decide, usando 2 cintas:
    - Copia  $w$  de la cinta 1 a la cinta 2. Tiempo:  $O(n)$ .
    - Posiciona el cabezal de la cinta 1 al principio. Tiempo:  $O(n)$ .
    - Compara en direcciones contrarias uno a uno los símbolos de ambas cintas hasta llegar a un blanco en ambas. Tiempo:  $O(n)$ .
    - Tiempo total:  $O(n) + O(n) + O(n) = 3 \cdot O(n) = O(n)$ .
- Ejemplo de lenguaje que no está en  $P$ :  $SAT$ , porque la única solución que se conoce es vía la tabla de verdad, la cual se chequea de forma exponencial.

## Clase $EXP$

- $EXP$  es la clase de los lenguajes decidibles en tiempo  $O(e^{poly(n)})$ .

## Tesis fuerte de Church-Turing

- Si un lenguaje es decidible en tiempo  $poly(n)$  por un modelo computacional realizable, también es decidible en tiempo  $poly(n)$  por una MT.
- Ejemplos de modelos computacionales realizables:
  - Máquinas RAM.
  - Programas Java.
  - Circuitos booleanos.
  - etc.
- Para la codificación de números cualquier base es válida excepto la unaria, porque la codificación unaria no es eficiente. Por ejemplo, el número 1000 se codifica como 1000 en base decimal, pero se codificaría como 111111111... (mil veces)

en base unaria, lo cual no es eficiente.

## Clase $NP$

- Un lenguaje  $L$  pertenece a la clase  $NP$  si existe una MT  $M$  tal que:
  - $w \in L$  si y solo si existe un certificado  $x$  tal que  $M$  acepta la tupla  $(w, x)$  en tiempo polinomial.
- En otras palabras, toda cadena  $w$  de  $L$  tiene un certificado  $x$  que permite **verificar de forma eficiente (polinomial)** que  $w$  pertenece a  $L$ .
- Si lo anterior se cumple, se dice que  $M$  es un **verificador eficiente** de  $L$  y que  $x$  es un **certificado sucinto** de  $w$ .

## Ejemplos de lenguajes que están en $NP$

- $SAT = \{\phi \mid \phi \text{ es una fórmula booleana sin cuantificadores, con } m \text{ variables y satisfactible}\}$ 
  - $SAT$  **no estaría** en  $P$  porque si  $\phi$  tiene  $m$  variables, entonces se deben chequear  $2^m$  posibles asignaciones de valores de verdad a las variables para encontrar una que satisfaga  $\phi$ .
  - $SAT \in NP$  porque dada una fórmula booleana  $\phi$  y una asignación de valores de verdad a sus variables  $A$ , se puede verificar en tiempo polinomial si  $A$  satisface  $\phi$ .
  - $SAT^C$  **no estaría** en  $NP$  porque para ver si  $\phi$  NO es satisfactible se debe hacer lo mismo que en el primer caso, es decir, chequear  $2^m$  posibles asignaciones de valores de verdad para asegurarnos de que no existe ninguna en absoluto que satisfaga  $\phi$ .
  - Por lo anterior se deduciría que  $SAT \notin CO-NP$ .
- Se cree que  $NP \neq CO-NP$ , pero no se ha podido demostrar, así como tampoco se ha podido demostrar que  $P \neq NP$ .

## Lenguajes $NPC$ y la clase $NPI$

### Introducción a $NPC$

- Asumiendo la conjetura  $P \neq NP$ , hay una forma de establecer que un lenguaje  $L \in NP$  no está en  $P$ .
- Lo anterior ocurre cuando se prueba que  $L$  es  $NPC$  (NP-completo).
- Los lenguajes  $NPC$  son los más difíciles de  $NP$ .

### Reducciones polinomiales

- Son igual que las reducciones generales, solo que la función de reducción  $f$  debe ser computada por una MT que trabaja en tiempo polinomial.
- Cumplen las mismas propiedades: son reflexivas, transitivas y no simétricas.
- Sirven para relacionar lenguajes.
- **Teorema 1:** Si  $L_1 \leq_p L_2$  y  $L_2 \in P$ , entonces  $L_1 \in P$ .
  - También, por contrarrecíproco, si  $L_1 \leq_p L_2$  y  $L_1 \notin P$ , entonces  $L_2 \notin P$ .

- **Teorema 2:** Si  $L_1 \leq_p L_2$  y  $L_2 \in NP$ , entonces  $L_1 \in NP$ .
  - También, por contrarrecíproco, si  $L_1 \leq_p L_2$  y  $L_1 \notin NP$ , entonces  $L_2 \notin NP$ .

## Definición de $NPC$

- $L \in NPC$  si y solo si:
  - $L \in NP$ .
  - Para todo  $L' \in NP$ ,  $L' \leq_p L$ . Es decir, todos los lenguajes de  $NP$  se reducen polinomialmente a  $L$ . Se dice que  $L$  es NP-difícil.
- Si  $L$  estuviera en  $P$ , entonces todos los lenguajes de  $NP$  estarían en  $P$ , y de esta manera se probaría que  $P = NP$ . Por ende, los lenguajes  $NPC$  no están en  $P$  a no ser que  $P = NP$ .
- $SAT$  es el problema más famoso que se sabe que está en  $NPC$ .

## Cómo poblar la clase $NPC$

1. Sea  $L \in NP$ .
2. Sea  $f$  una función de reducción polinomial de  $SAT$  a  $L$ .
3. Sea  $g$  la función de reducción polinomial obtenida componiendo la reducción polinomial  $f_i$  de cada  $L_i \in NP$  con la reducción  $f$  de  $SAT$  a  $L$ .
4. Lo anterior vale para todos los lenguajes de  $NP$ , por lo tanto  $L$  es  $NPC$ .
5. En resumen:
  - i.  $L_1 \in NPC$
  - ii.  $L_2 \in NP$
  - iii.  $L_1 \leq_p L_2$
  - iv.  $L_2 \in NPC$

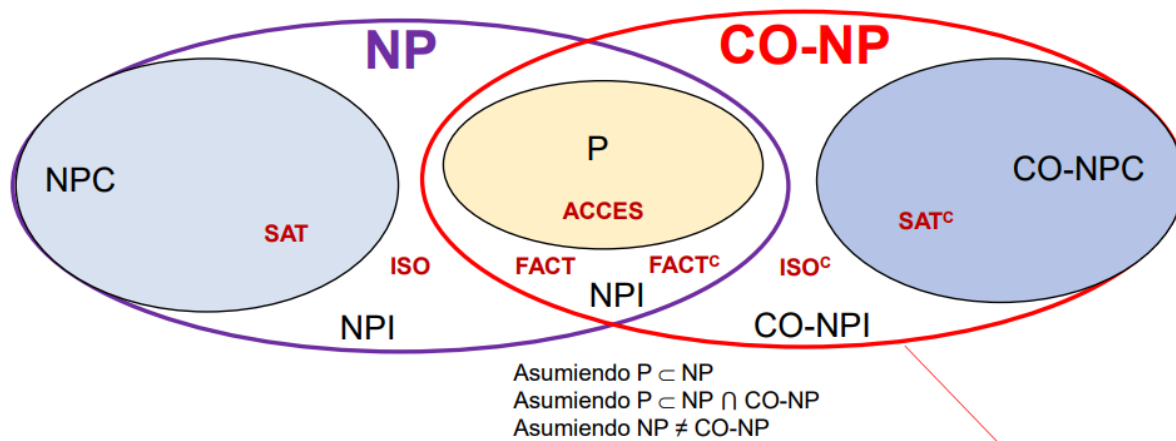
## Dos características clave de los lenguajes $NPC$

1. Por definición, par todo par de lenguajes  $L_1, L_2 \in NPC$  se cumple que  $L_1 \leq_p L_2$  y  $L_2 \leq_p L_1$ .
  - i. Sin embargo, la función de reducción de la segunda reducción no necesariamente es la función inversa de la  $f$  de la primera reducción.
  - ii. A pesar de esto, se conjetura que todos los lenguajes  $NPC$  cumplen esta propiedad llamada p-isomorfismo, es decir, para todo par de lenguajes  $L_1, L_2 \in NPC$  existe una función de reducción polinomial  $f$  de  $L_1$  a  $L_2$  tal que su inversa  $f^{-1}$  es también una función de reducción polinomial de  $L_2$  a  $L_1$ .
  - iii. Si esta conjetura logra probarse, entonces se prueba  $P \neq NP$ .
2. Todos los lenguajes  $NPC$  conocidos son **densos**.
  - i. Un lenguaje es **denso** si para todo  $n$  tiene  $exp(n)$  cadenas de longitud a lo sumo  $n$ .
  - ii. Un lenguaje es **disperso** en caso contrario: para todo  $n$  tiene  $poly(n)$  cadenas de longitud a lo sumo  $n$ .
  - iii. Se conjetura que todos los lenguajes  $NPC$  son densos.
  - iv. Si existe un lenguaje  $NPC$  que es disperso, entonces se prueba  $P = NP$ .

## Clase $NPI$

- Los lenguajes de  $NPI$  no son ni tan fáciles como los de  $P$  ni tan difíciles como los de  $NPC$ . Están en el medio.
- Un lenguaje que podría estar en  $NPI$  es el lenguaje de los grafos isomorfos. Dos grafos son isomorfos si son idénticos salvo por el nombre de sus nodos. Se define como:
  - $ISO = \{(G_1, G_2) \mid G_1 \text{ y } G_2 \text{ son grafos isomorfos}\}$
  - Está en  $NP$  porque los certificados sucintos son permutaciones de  $V$ , el conjunto de nodos de  $G_1$ , que permiten verificar de forma eficiente si  $G_1$  y  $G_2$  son isomorfos.
  - No estaría en  $P$  porque en el peor caso hay que probar todas las permutaciones de  $V$ .
  - No estaría en  $NPC$  porque no se ha encontrado un lenguaje  $NPC$  tal que se reduzca polinomialmente a  $ISO$ .
  - Su complemento no estaría en  $NP$  porque no se ha encontrado ningún certificado sucinto que permita verificar de forma eficiente que dos grafos no son isomorfos.

## Diagrama de Venn de la jerarquía temporal



### Jerarquía de lenguajes (de menor a mayor dificultad)

- 1) P
- 2)  $(NP \cap CO-NP) - P$
- 3) NPI - CO-NP
- 4) NPC
- 5) CO-NPI - NP
- 6) CO-NPC

Las cadenas de los lenguajes de CO-NP no tendrían certificados sucintos. Por ejemplo, para verificar si una fórmula booleana no tiene una asignación que la satisface, hay que chequear todas las posibles asignaciones, las cuales suman una cantidad exponencial.

## Complejidad espacial

## Introducción

- Se consideran MT tales que la cinta de entrada es readonly. Las demás son cintas de trabajo read/write.
- Una MT ocupa espacio  $S(n)$  si y solo si, en todas sus cintas de trabajo (no la de entrada) ocupa a lo sumo  $S(n)$  celdas, siendo  $n = |w|$  la longitud de la cadena de entrada.

- La cinta de entrada readonly permite espacios menores que  $O(n)$ .
- Una MT  $M$  que ocupa espacio  $S(n)$  puede no detenerse, pero dada  $M$ , siempre existe una MT  $M'$  equivalente que ocupa espacio  $S(n)$  y siempre se detiene.
  - Por ejemplo, una MT con 1 cinta de entrada readonly y 1 cinta de trabajo read/write entra en loop si hizo más de  $n \cdot S(n) \cdot |Q| \cdot |\Gamma|^{S(n)} = O(c^{S(n)})$  pasos.
  - Por lo tanto, tiempo  $T(n)$  implica espacio  $S(n)$ , y espacio  $S(n)$  implica tiempo  $O(c^{S(n)})$ .

## Clase $SPACE(S(n))$

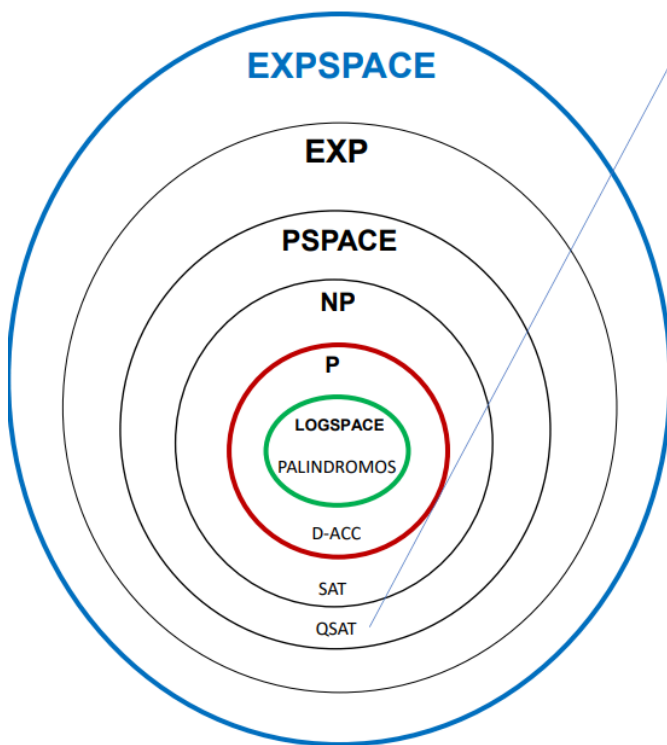
- Un lenguaje  $L$  pertenece a la clase  $SPACE(S(n))$  si y solo si existe una MT  $M$  que decide  $L$  ocupando espacio  $O(S(n))$ .

## Jerarquía espacial

- $LOGSPACE$  es la clase de los lenguajes aceptados en espacio  $O(\log_2(n))$ .
- $PSPACE$  es la clase de los lenguajes aceptados en espacio  $poly(n)$ .
- $EXPSPACE$  es la clase de los lenguajes aceptados en espacio  $exp(n)$ .
- Espacio  $S(n)$  implica tiempo  $O(c^{S(n)})$ , con  $c$  constante. Así, si en particular una MT  $M$  ocupa espacio  $\log_2(n)$ , entonces  $M$  tarda tiempo  $O(c^{\log_2(n)}) = O(n^{\log_2(c)}) = O(n^k)$ , con  $k$  constante. Por lo tanto,  $LOGSPACE \subseteq P$ .
- Los problemas tratables en espacio son los que pertenecen a la clase  $LOGSPACE$ .
- Existe una clase  $NLOGSPACE$  homóloga a la clase  $NP$  pero con espacio en lugar de tiempo. También se cumple  $NLOGSPACE \subseteq P$ .

# Diagrama de Venn de la jerarquía espacio-temporal

## Jerarquía espacio-temporal



- QSAT está entre los lenguajes más difíciles de PSPACE.
- En efecto, **QSAT es PSPACE-completo**, todos los lenguajes de PSPACE se reducen polinomialmente a QSAT.
- QSAT es una instancia de un problema más general: **la búsqueda de una estrategia ganadora en una competencia entre dos jugadores  $J_1$  y  $J_2$  (ajedrez, damas, go, hexágono, geografía, etc.):**

¿Existe una jugada 1 de  $J_1$  tal que para toda jugada 1 de  $J_2$  existe una jugada 2 de  $J_1$  tal que para toda jugada 2 de  $J_2$  existe una jugada 3 de  $J_1$  tal que para toda jugada 3 de  $J_2$  ... el jugador  $J_1$  gana?

En el caso particular de QSAT se puede plantear que  $\exists$  busca que  $\phi$  sea verdadera y  $\forall$  que  $\phi$  sea falsa.

$$\phi = [\exists x_1 \forall x_2 \exists x_3 \forall x_4 \exists x_5 \forall x_6 \dots : \Psi(x_1, x_2, x_3, x_4, x_5, x_6, \dots)]$$